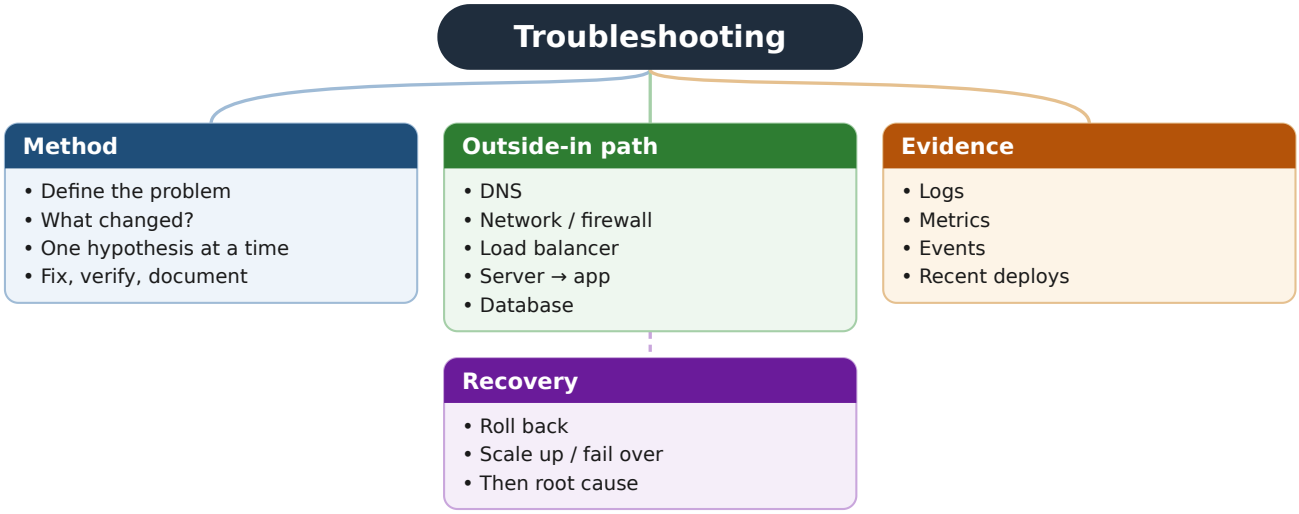
17. Troubleshooting Playbooks

In this section:



Universal method: 1) Define the problem exactly. 2) What changed? 3) Check from the outside in (DNS → network → load balancer → server → app → database). 4) Read logs and metrics. 5) Form one hypothesis, test it. 6) Fix, verify, document.

Scenario	Investigate in this order
Website is down	DNS resolves? (`dig`) → port open? (`curl -v`, `nc -zv`) → load balancer target health → server up? → service running? (`systemctl status`) → app logs → recent deploy? → database reachable?
DNS not resolving	`dig name` vs `dig @8.8.8.8 name` → record exists/correct in hosted zone? → TTL/caching → `/etc/resolv.conf` → typo / wrong domain
Server can't reach the internet	IP + gateway (`ip a`, `ip route`) → `ping 8.8.8.8` (network) vs `ping google.com` (DNS) → AWS: route to IGW/NAT, NAT in public subnet, SG/NACL outbound
EC2 unreachable	Instance state + status checks → public IP/EIP → SG inbound 22/80 from your IP → NACL → route table IGW → key/username → SSM Session Manager as a fallback
High CPU	`top` / `htop` → which process? → app logs → traffic spike or loop/bug? → scale out or fix → set alerts
Disk is full	`df -h` (and `df -i`) → `du -sh` to find big directories → logs, journal, Docker images, temp files → clean safely → add logrotate/alerts/bigger disk
App can't connect to database	Correct host/port/credentials (from secrets)? → `nc -zv db 5432` → SG allows app SG → DB running/available → max connections → TLS settings
Docker container keeps restarting	`docker ps -a` exit code → `docker logs` → `docker inspect` (OOMKilled) → env/config/volumes → run interactively to test
Pod CrashLoopBackOff	`kubectl describe pod` (Events, Last State) → `kubectl logs --previous` → missing ConfigMap/Secret/env → wrong command → probe killing it → memory limit

Scenario	Investigate in this order
Deployment failed / not rolling out	`kubectl rollout status` → new pods' status (ImagePull? Crash? Pending?) → `kubectl rollout undo` if needed
Pipeline failed	Failing stage → first error in logs → code vs environment vs credentials vs network → reproduce locally → fix and re-run
Terraform apply failed	Read the exact error → credentials/region (`aws sts get-caller-identity`) → state lock → resource exists → quotas → dependency order → re-plan
Slow application	Latency dashboard → which endpoint → traces (which step) → DB slow queries → CPU/memory saturation → external dependency

Golden rules: change one thing at a time; keep notes; restore service first (rollback), then find the root cause; write a short blameless post-incident review.

REMEMBER

- The first question is always **"What changed?"**
- Change one thing at a time and write down what you did.